

Semistructured Data Integration (I)

TECHNOLOGIES FOR INFORMATION SYSTEMS

Letizia Tanca

Politecnico di Milano

Again recall the new application context

- A (possibly large) number of data sources
- Heterogeneous data sources (use of wrappers)
- Different levels of data structure
 - Databases (relational, OO...)
 - Semi-structured data sources (XML, HTML, more markups ...)
 - Unstructured data sources (text, multimedia etc...)
- Different terminologies and different operational contexts
- Time-variant data (e.g. WEB)
- Mobile, transient data sources

SEMISTRUCTURED DATA

FOR THIS DATA THERE IS SOME FORM OF STRUCTURE, BUT IT IS **NOT** AS

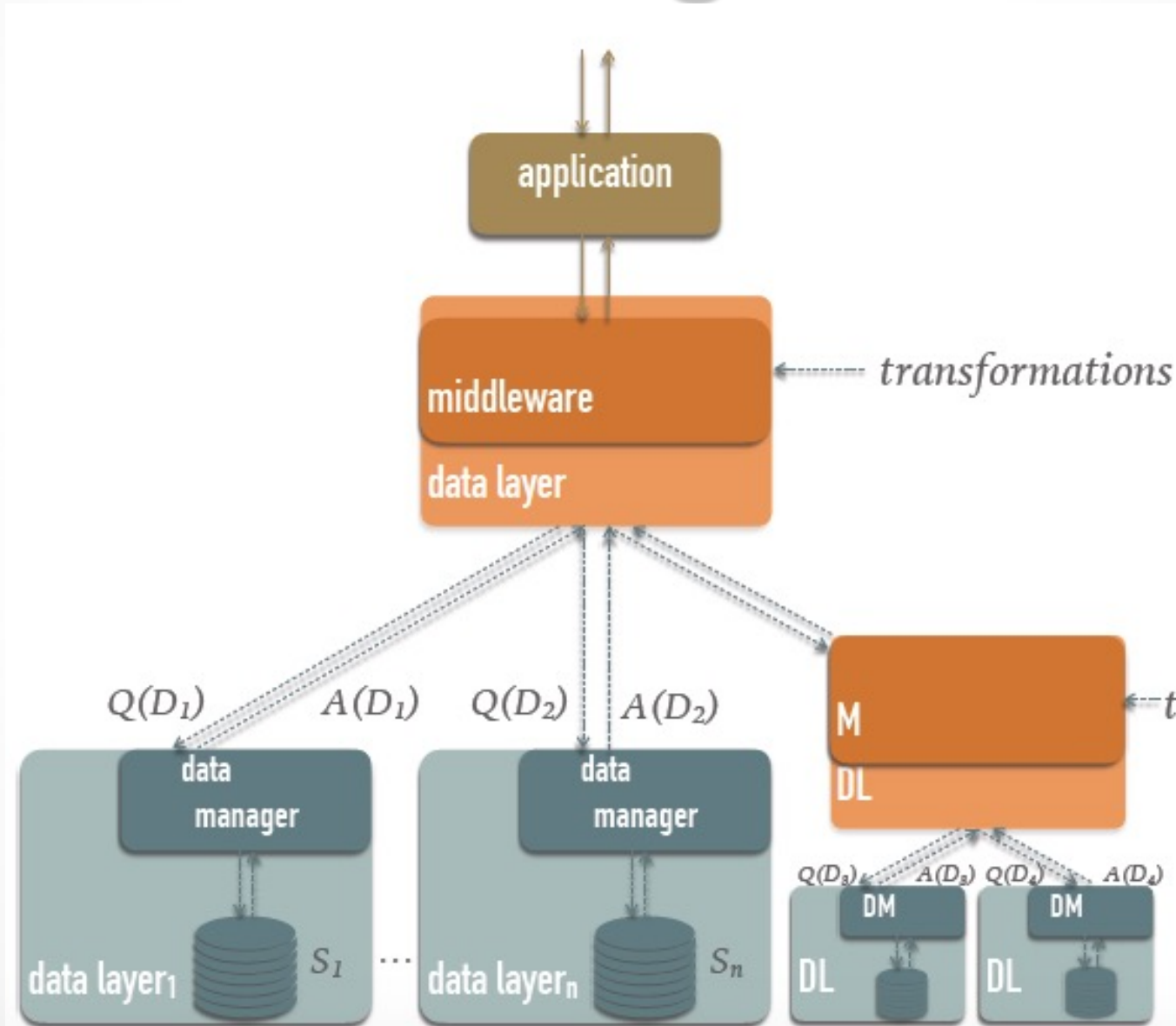
- **PRESCRIPTIVE**
- **REGULAR**
- **COMPLETE**

AS IN TRADITIONAL DBMSs

EXAMPLES

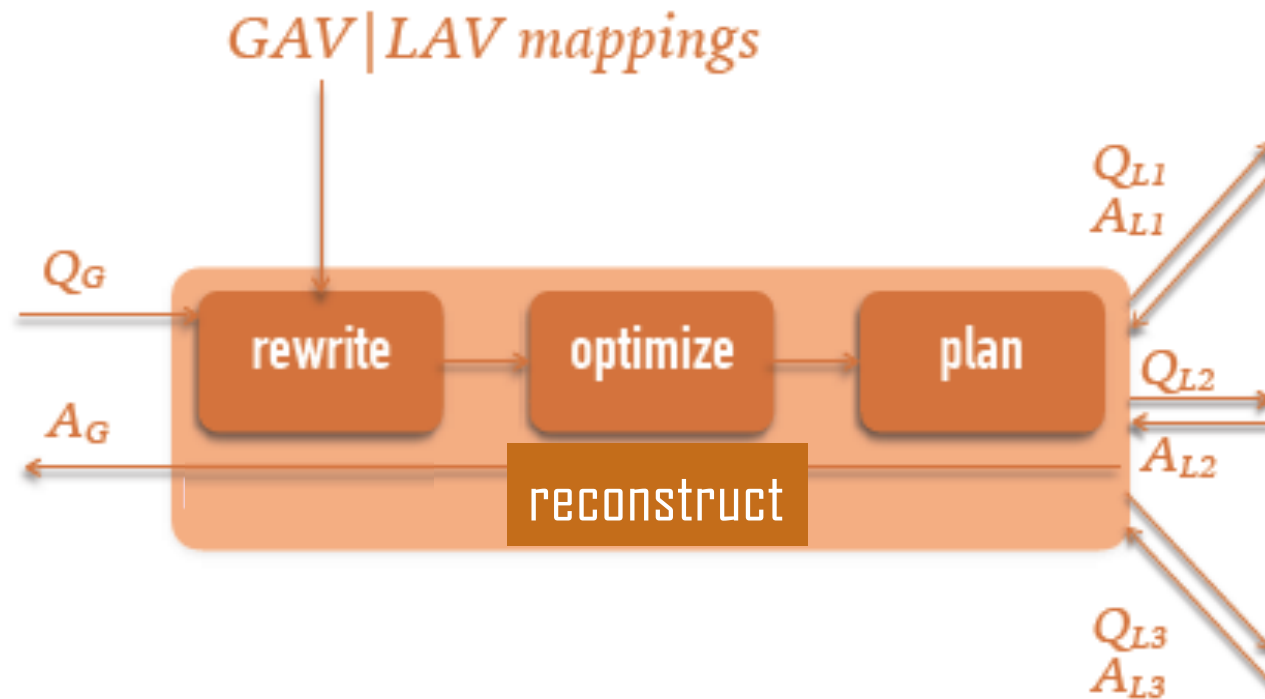
- **WEB DATA (HTML)**
- **XML DATA**
- **VARIOUS CLOUD DATA MODELS**

Our general framework for Data Integration



(Alvaro Fernandez,
EDBT Summ.
School 2017)

A closer look at the middleware



(from Alvaro Fernandez, EDBT Summ.School 2017)

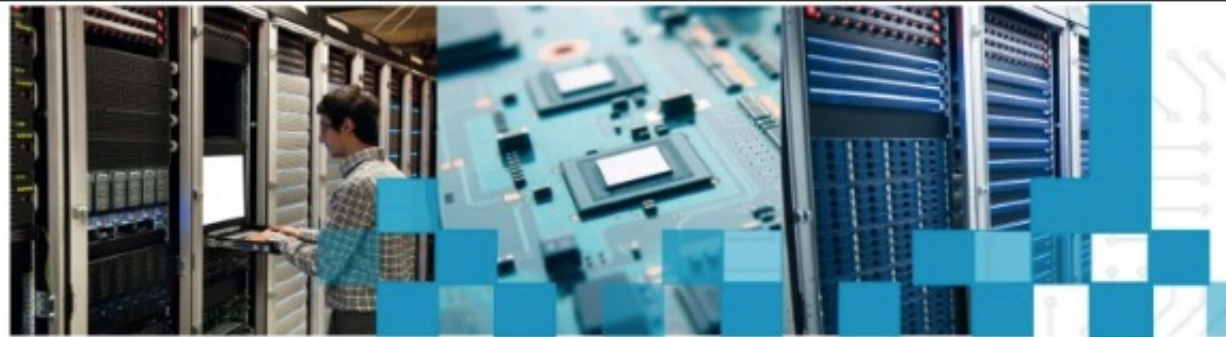
AN EXAMPLE OF SEMISTRUCTURED DATA

Prof. Letizia Tanca

POLITECNICO DI MILANO



DIPARTIMENTO DI
ELETTRONICA,
INFORMAZIONE
E BIOINGEGNERIA



[Letizia Tanca](#) [Books](#) [Projects](#) [Publications](#) [Research Interests and Thesis Topics](#) [Teaching](#) [Contacts](#)

Letizia Tanca



Big Data, and its 5 Vs – volume, velocity, variety, veracity, and value – have been talked about a lot. In their **ACM SIGMOD blog post**, Letizia Tanca is writing about the **Double V for Big Data: Wisdom**. Reciting an extract from her blog: *...not only we want to*

Recent Comments

Archives

Categories

• No categories

Meta

- [Log in](#)
- [Entries RSS](#)
- [Comments RSS](#)
- [WordPress.org](#)

SEMISTRUCTURED DATA:

a page produced from a database

[ITA](#) [ENG](#)

[SCHOOL@DEIB](#) [I4.0](#) [HONOURS PROGRAM](#) [PEoPL@DEIB](#) [EVENTI DEIB](#) [INTRANET](#)

**POLITECNICO
MILANO 1863**
DIPARTIMENTO DI ELETTRONICA
INFORMAZIONE E BIOINGEGNERIA

[HOME](#) [NOTIZIE ED EVENTI](#) [CHI SIAMO](#) [RICERCA](#) [INDUSTRIA](#) [PERSONALE](#) [RISORSE](#) [DIDATTICA](#)

» Personale

Prof. TANCA LETIZIA

Professore Ordinario





Sede: Edificio 20
Piano: 1°
Ufficio: 050
Tel.: 3531

 letizia.tanca@polimi.it



Area di ricerca:
» [Informatica](#)

Linea di ricerca:
» [Dati, web e società](#)



Pagina personale
<http://tanca.faculty.polimi.it>

... or an e-commerce site...



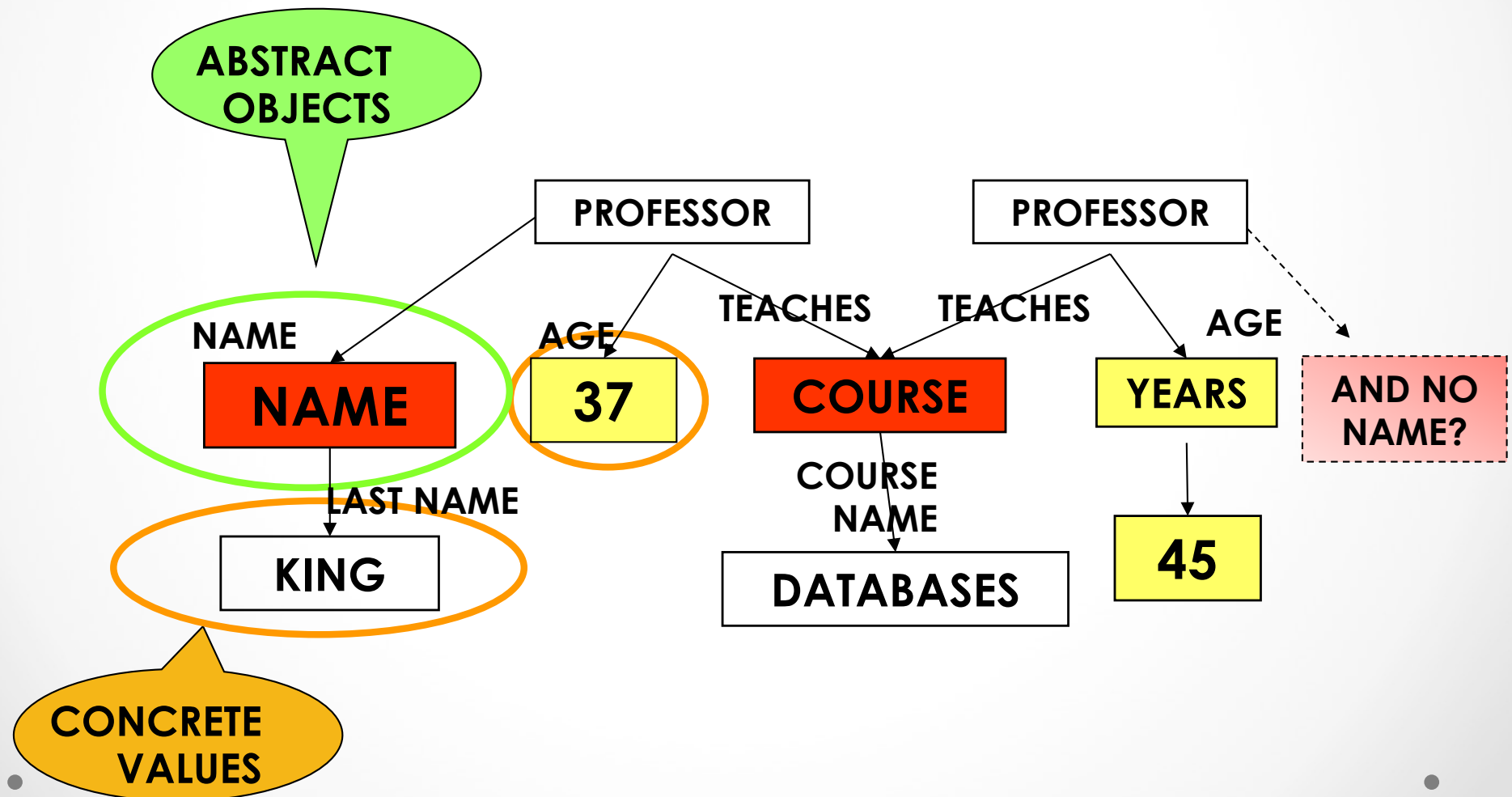
SEMISTRUCTURED DATA MODELS

BASED ON TEXT AND/OR DATA STRUCTURES AS:

- TREES
 - GRAPHS
- with
- LABELED NODES
 - LABELED ARCS
 - BOTH

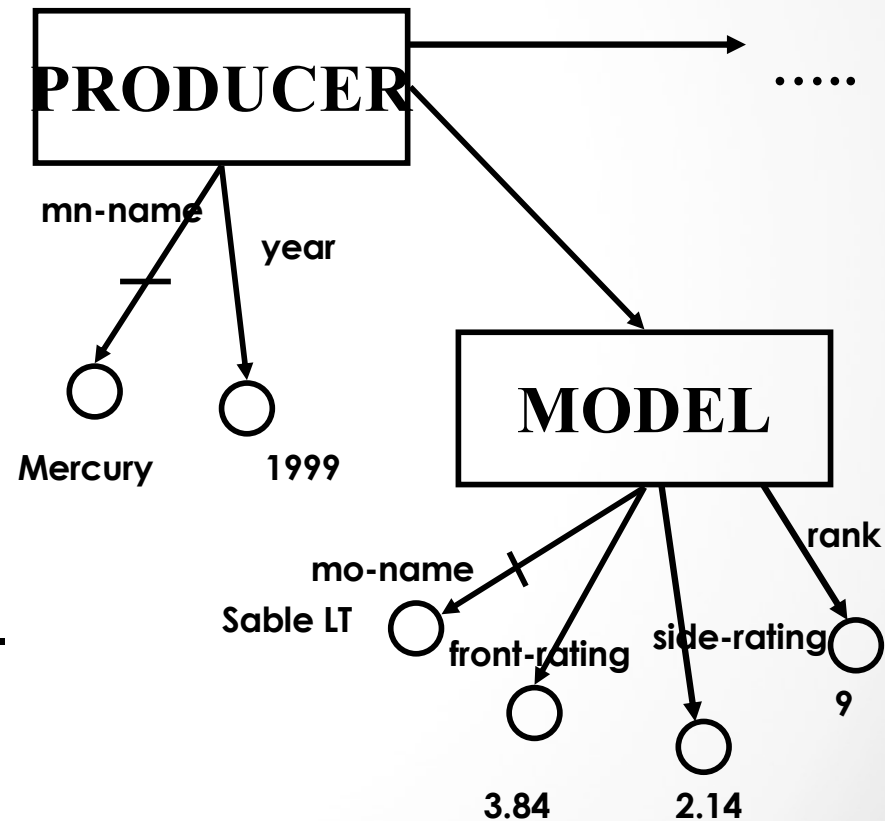
CAUTION: SEMISTRUCTURED DATA MODELS ARE ALL DIFFERENT AND DO NOT LEND THEMSELVES TO EASY INTEGRATION !!!

A COMMON GRAPH-BASED REPRESENTATION, WHERE THE **IRREGULAR DATA STRUCTURE** APPEARS VERY CLEARLY

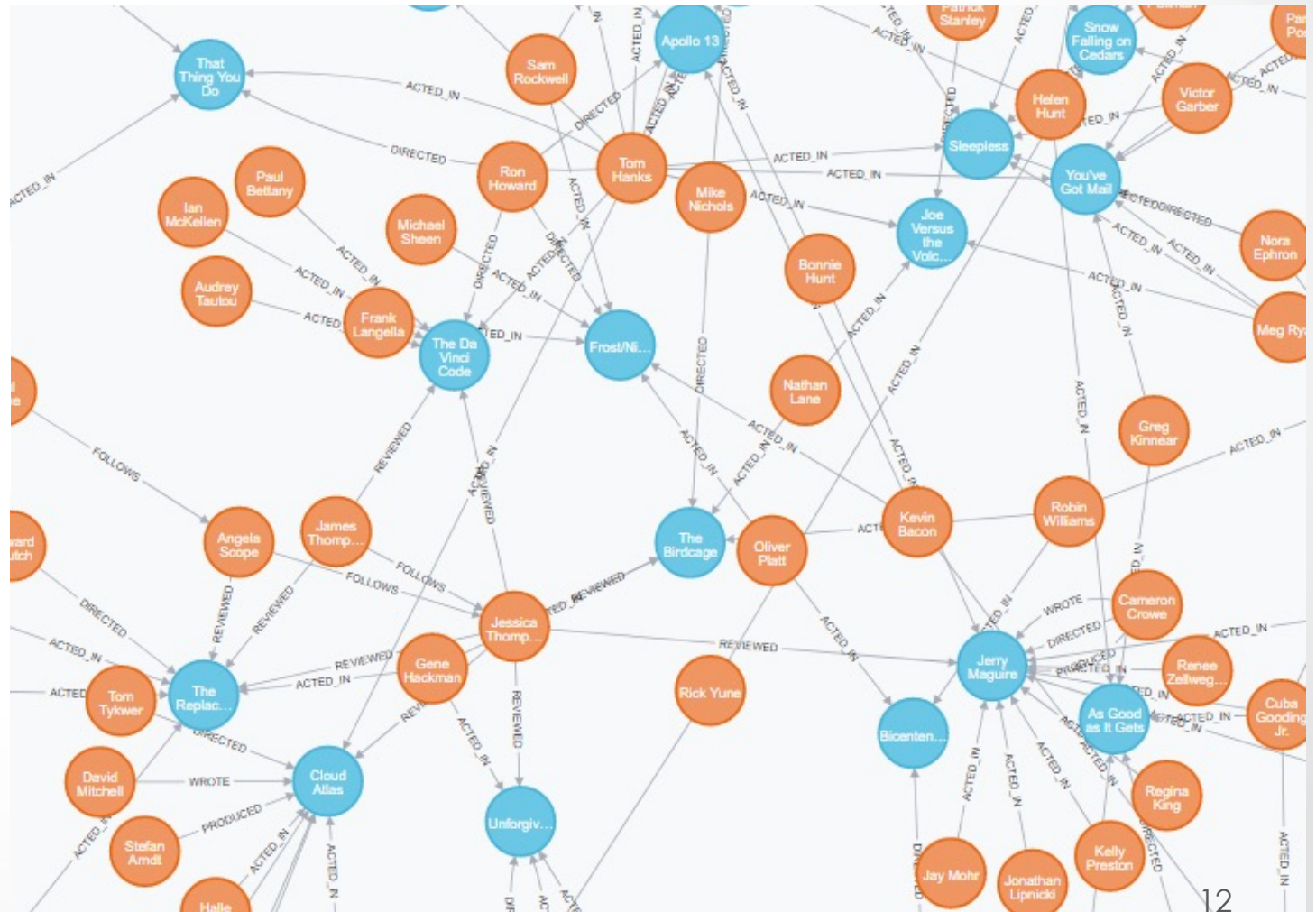


A SIMPLE XML DOCUMENT WITH ITS GRAPH-BASED REPRESENTATION

```
<producer>
  <mn-name>Mercury</mn-name>
  <year>1999</year>
  <model>
    <mo-name>Sable LT</mo-name>
    <front-rating>3.84</front-rating>
    <side-rating>2.14</side-rating>
    <rank>9</rank>
  </model>
  .....
</producer>
```



One of the best organized graph-based data models:



INFORMATION SEARCH IN SEMISTRUCTURED DATABASES

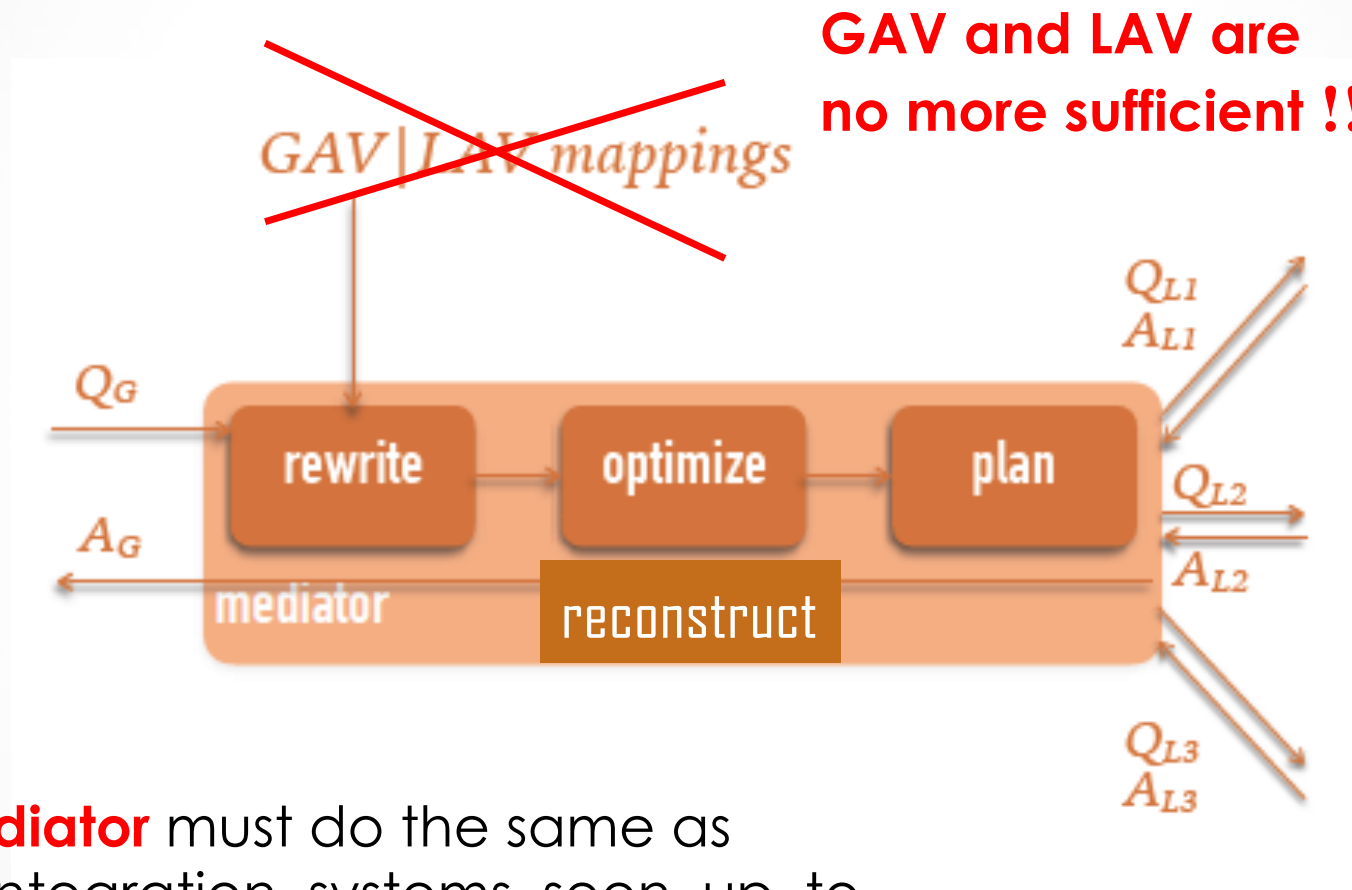
- WE WOULD LIKE TO:

- INTEGRATE
- QUERY
- COMPARE

DATA WITH DIFFERENT STRUCTURES **ALSO WITH SEMISTRUCTURED DATA**, JUST AS IF THEY WERE ALL STRUCTURED

- AN OVERALL DATA REPRESENTATION SHOULD BE **PROGRESSIVELY BUILT**, AS WE **DISCOVER** AND **EXPLORE** NEW INFORMATION SOURCES

THE MIDDLEWARE SHOULD CONTAIN MEDIATORS



A **mediator** must do the same as the integration systems seen up to now, but this time **the problem is much more complex**

MEDIATORS must do many different things

The term mediation includes:

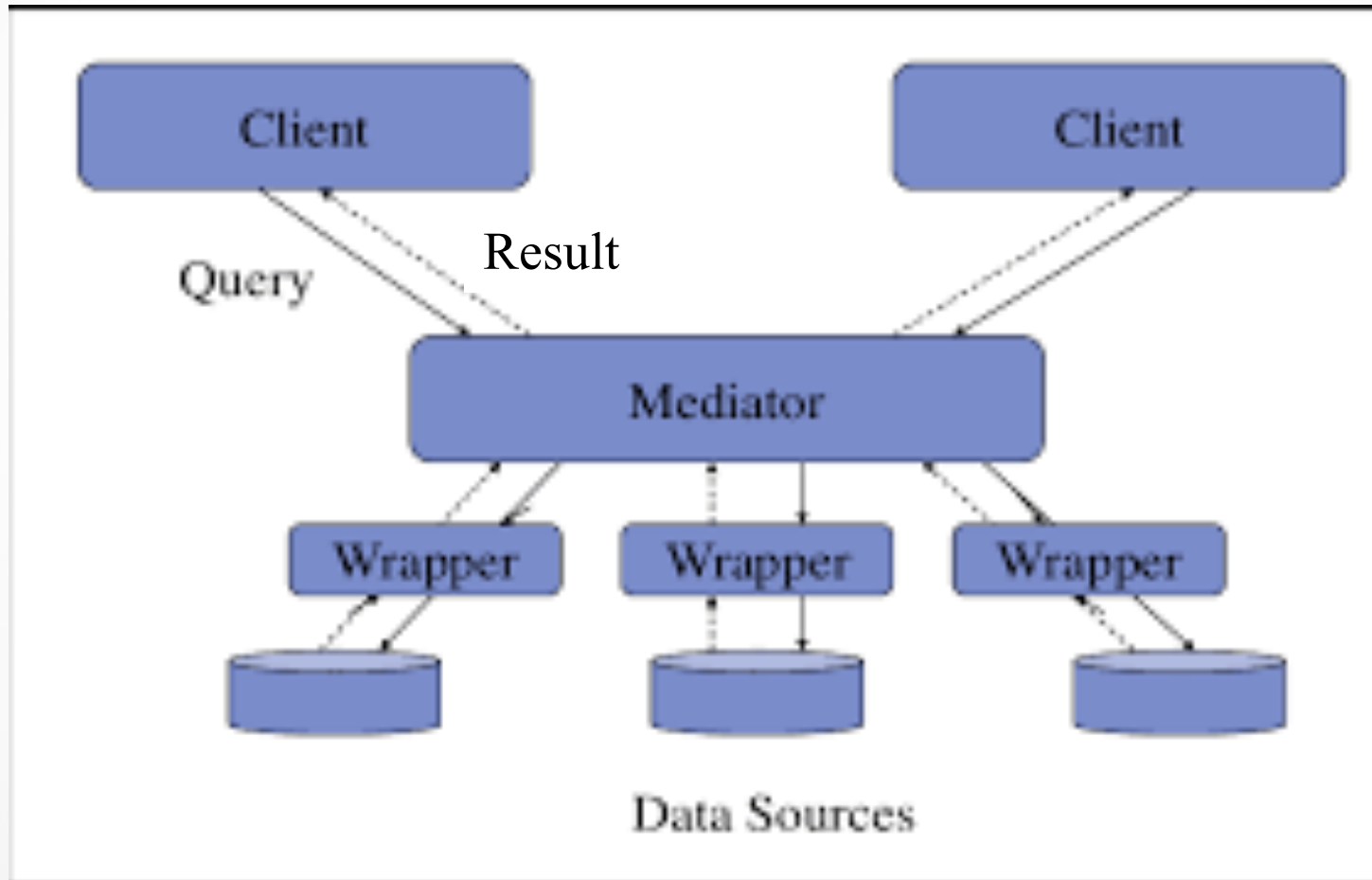
- the **processing** needed to make the interfaces work
- the **knowledge structures** that drive the transformations needed to transform data to information
- any **intermediate storage** that is needed (Wiederhold)

Problem:

**each different domain needs a mediator
appropriately designed to “understand” its semantics**

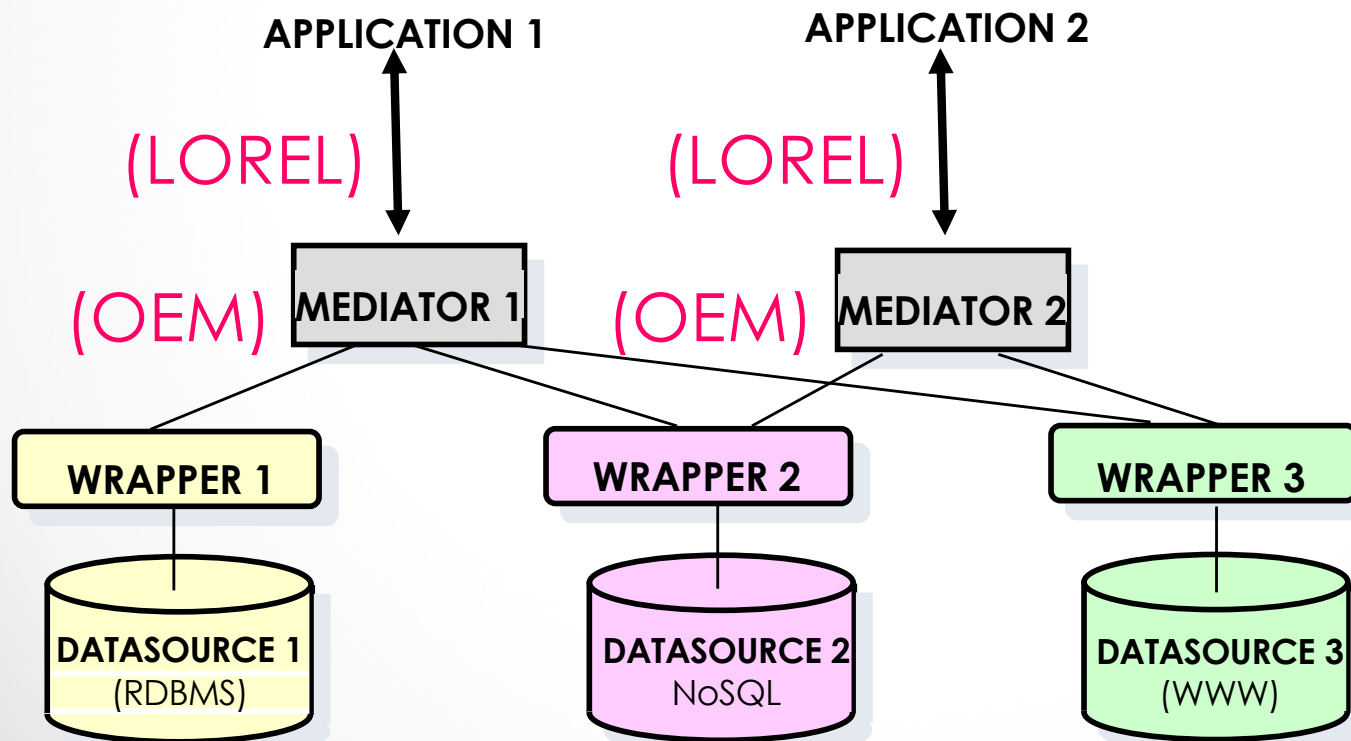
!!!

Mediation-based systems



EXAMPLE: TSIMMIS

- The first system that was based on the **mediator/wrapper paradigm**
- Proposed already in the 90's at **Stanford university**



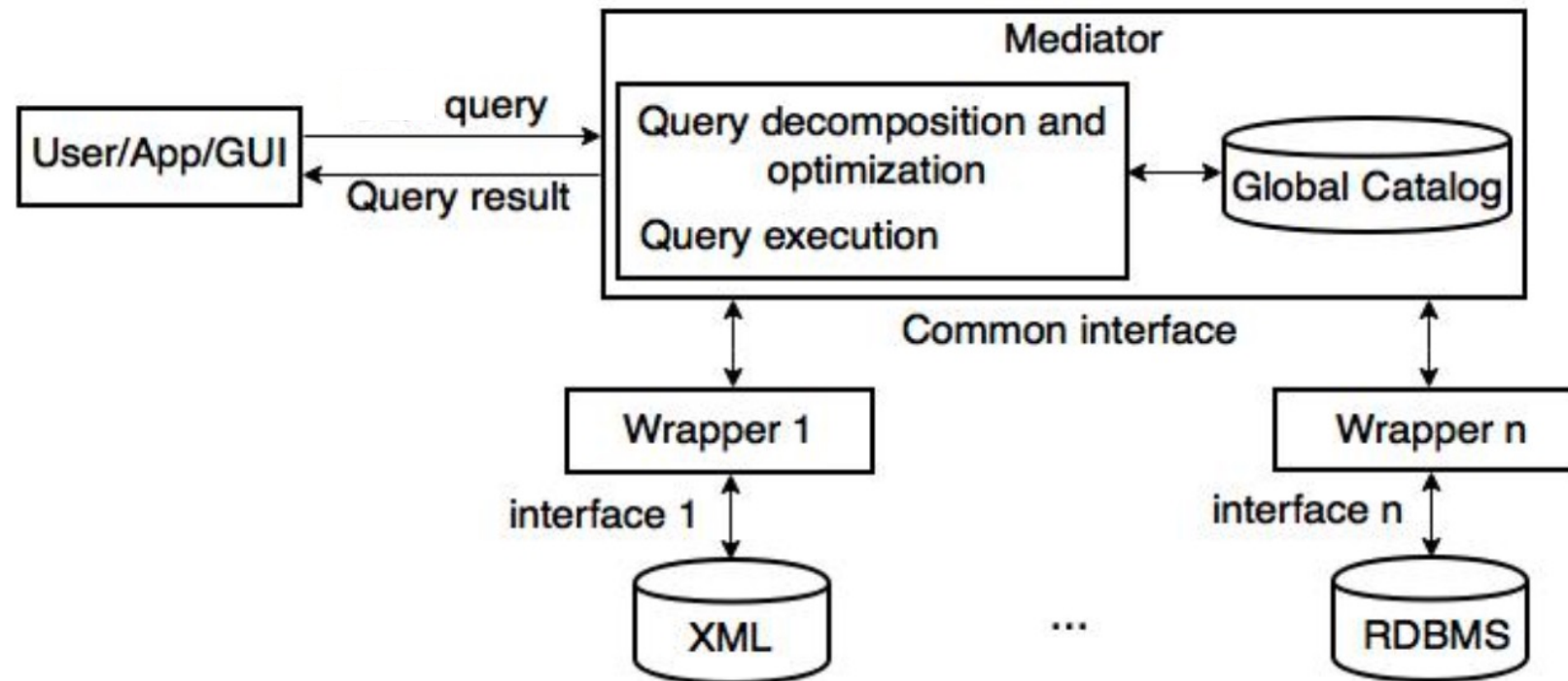
TSIMMIS:

A mediator-based approach

- **OEM:** A UNIQUE, GRAPH-BASED INTERNAL DATA MODEL MANAGED BY THE MEDIATOR
- WRAPPERS FOR THE **MODEL-TO-MODEL TRANSLATIONS**
- QUERY POSED TO THE MEDIATOR IN THE **LOREL** LANGUAGE

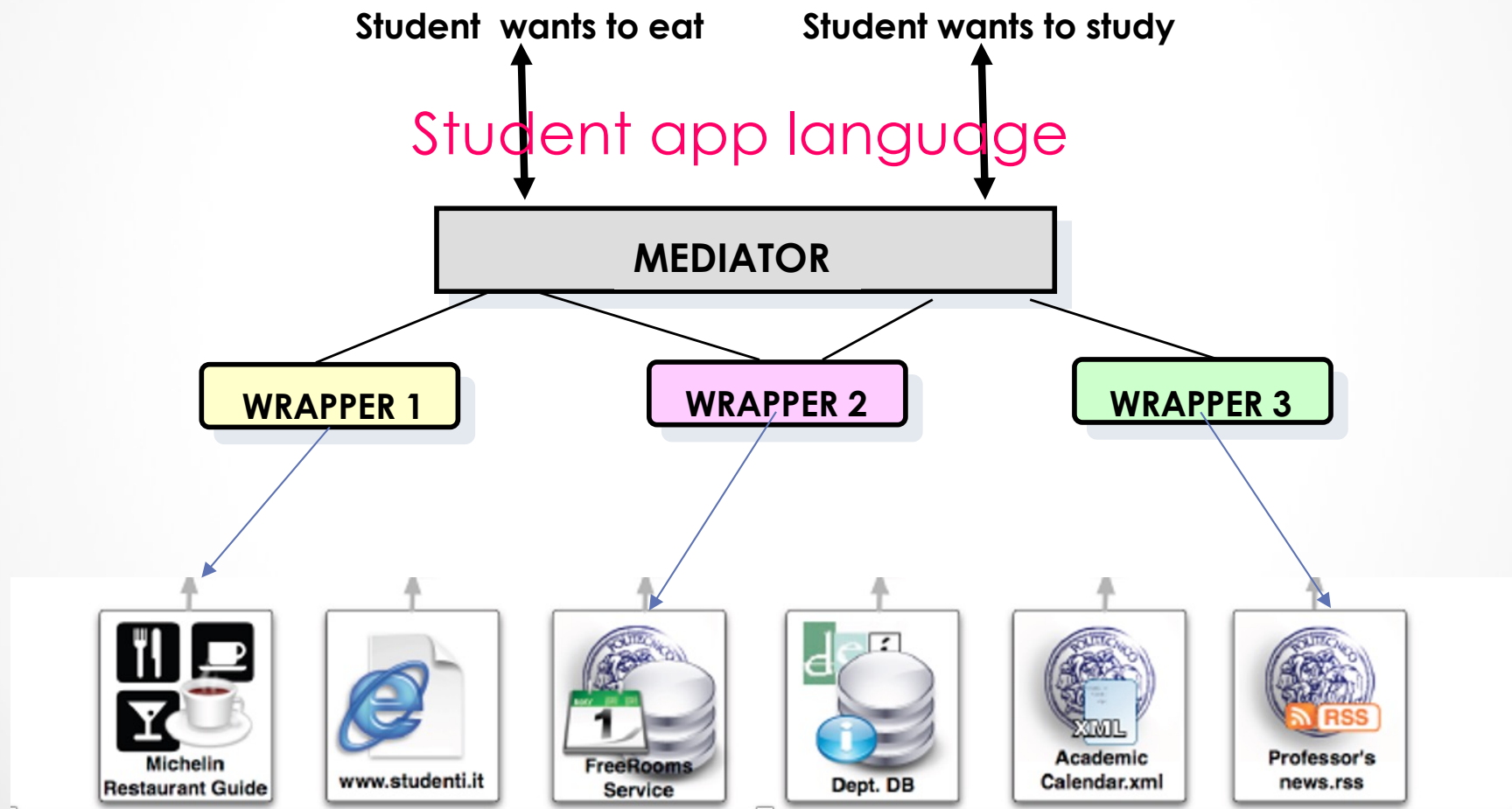
THE MEDIATOR **KNOWS** THE SEMANTICS
OF THE APPLICATION DOMAIN

General Mediator-based Architecture



In this architecture there is a clear separation of concerns: the mediator deals with data source distribution while the wrappers deal with data source heterogeneity and autonomy. This is achieved by using **a common language** between mediator and wrappers, and the translation to the data source language is done by the wrappers. Each data source has an associated wrapper that exports information about the source schema, data and query processing capabilities

An example of the use of mediators



A semistructured integration example

An important TV director wants to integrate the database of his company, containing only data about TV shows, with the data of another database concerning a wider variety of shows. The final integrated database must contain information related exclusively to TV shows coming from both the sources.

The first source is a standard relational database:

SHOW (Title, Date, Type)
CASTMEMBER (Code, Firstname, Lastname, Address, FeePerHour, AgentCode)
CAST (ShowTitle, ShowDate, CastMemberCode, AppearanceDuration)
AGENT (Code, Firstname, Lastname, Company, PhoneNumber)
ROLE (Name, Description)
CASTROLE (CastMemberCode, RoleName)

Remarks:

- The duration of the show is expressed in hours.
- The fees are expressed in dollars.
- Dates are expressed as yyyy-mm-dd.
- The possible roles are guest, stable-member and technician.
- Type of the show can be TvShow, Theater, ...

The second source is that used in the web information system of the TV company, which relies on an XML database and contains only information about TV shows:

```
<!ELEMENT TvShowsDB (TvShow*)>
<!ELEMENT TvShow (Anchor+, Schedule)>
<!ELEMENT Anchor EMPTY>
<!ELEMENT Schedule (Day+)>
<!ELEMENT Day (Guest+)>
<!ELEMENT Guest EMPTY>
<!-- ATTLIST -->
<!-- TvShow -->
  Title CDATA #REQUIRED
  Edition CDATA #REQUIRED
  Duration CDATA #REQUIRED
<!-- Anchor -->
  Name CDATA #REQUIRED
  EngagementFee CDATA #REQUIRED
  AgentPhoneContact CDATA #IMPLIED
<!-- Day -->
  Date CDATA #REQUIRED
  Special (0|1) "0"
<!-- Guest -->
  Name CDATA #REQUIRED
  EngagementFee CDATA #REQUIRED
  Address CDATA #IMPLIED
```

An example of a valid XML file compliant with the schema is:

```
<TvShowsDB>
  <TvShow Title="The David Letterman Show" Edition="2016/2017" Duration="1">
    <Anchor Name="David$Letterman" EngagementFee="60,000"/>
    <Schedule>
      <Day Date="2016-09-14" Special="0">
        <Guest Name="Mike$Tyson" EngagementFee="200,000"/>
      </Day>
      <Day Date="2016-09-21">
        <Guest Name="Hillary$Clinton" EngagementFee="30,000"/>
        <Guest Name="Donald$Trump" EngagementFee="30,000"/>
      </Day>
    </Schedule>
  </TvShow>
</TvShowsDB>
```

Remarks:

- The duration of the show is expressed in hours.
- The fees are expressed in dollars.
- The fee of an anchor or a guest may change for each edition of the show or date.
- Dates are expressed as yyyy-mm-dd.
- The anchor and guest names are in the form "*Firstname\$Lastname*".
- Suppose that there are no duplicates in the names of anchors and guests.

NOTE: you can suppose that the people in the two data sources are disjoint, and that the shows in the two data sources are disjoint.

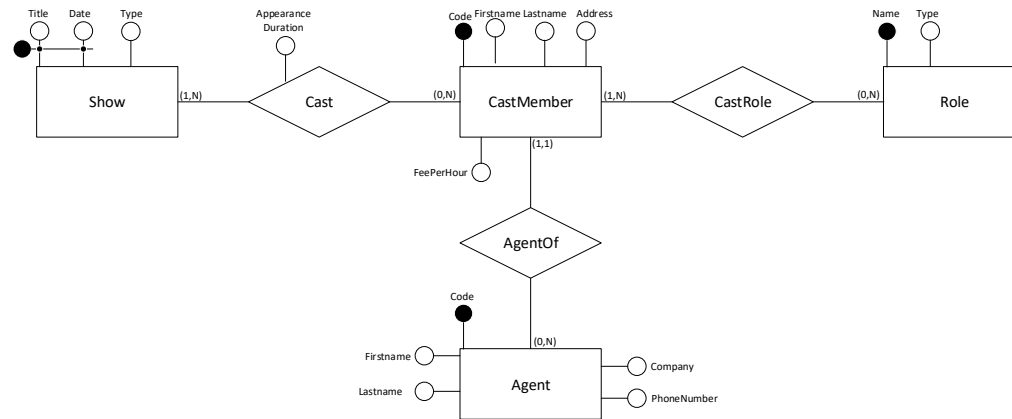
Integrate the two data sources, following these steps:

- 1) Source schema reverse engineering
- 2) Schema integration
 - a. Related concept identification + Conflict analysis and resolution
 - b. Production of the global conceptual schema
 - c. Conceptual to logical translation of the global schema
- 3) Query formulation and mapping definition
 - a. Write in SQL the following query Q on the global schema: "*Select first name, last name and address of the cast members of TV shows that have been on air in special occasions*"
 - b. GAV mapping definition (only for the tables required to answer the query Q at point 3a)
 - c. Show the rewriting of Q on the data sources in SQL

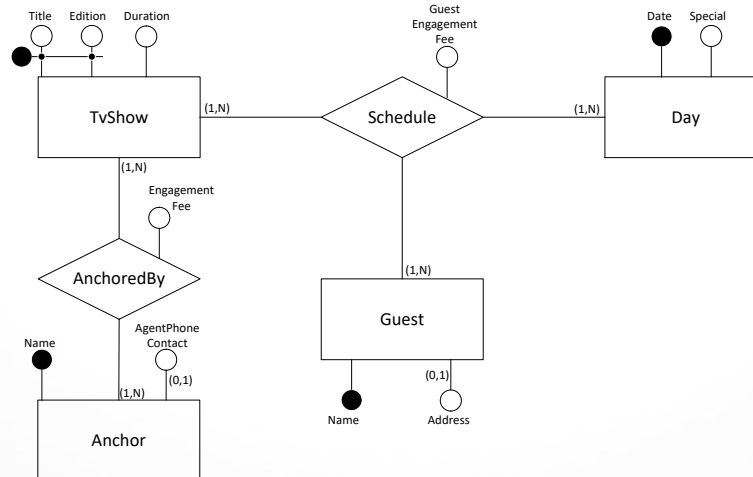
SOLUTION

1. Source schema reverse engineering

DS1



DS2



DS2 Logical schema

TVSHOW (Title, Edition, Duration)

ANCHOR (Name, AgentPhoneContact*)

GUEST (Name, Address*)

DAY (Date, Special)

ANCHORED BY (TvShowTitle, TvShowEdition, AnchorName, EngagementFee)

SCHEDULE (TvShowTitle, TvShowEdition, Date, GuestName, GuestEngagementFee)

***NOTE:** In this course we do not study how to implement wrappers. Therefore, we suppose the existence of wrappers guaranteeing the correspondence between the XML source and its relational counterpart, without entering in details.*

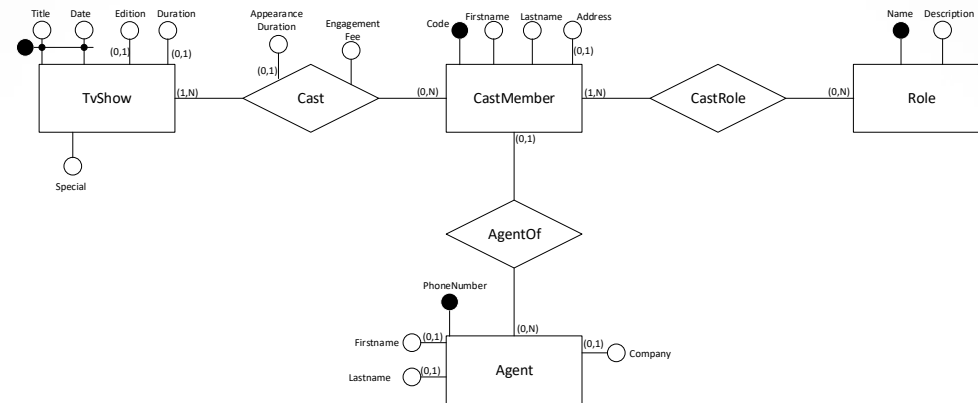
2. Schema integration

Do note that the exercise does not request to build an integrated schema capturing all the data coming from the two sources, but only the data in the two sources that deal with tv shows.

2a) Related concept identification + Conflict analysis and resolution

DS1	DS2	Conflict	Solution
Show	TvShow	Name Conflicts	
		- Entity name	TvShow (only tv shows are required)
		Structure conflicts	
		- A tuple for each date → a tuple for each edition, with different dates represented in the Day entity	Use a tuple for each date (more general)
		Key conflict	
		- Title+Date → Title+Edition	Title+Date
CastMember	Anchor/ Guest	Name conflicts	
		- Entity name	CastMember
		Data semantics conflicts	
		- FeePerHour → EngagementFee	EngagementFee
		Structure conflicts	
		- An attribute for the last name and one for the first name → just one attribute "Firstname\$ Lastname"	Two attributes
		- Fee unique for each person → Fee varies with edition/appearance	Fee varies with edition/appearance
		- Agent is an entity → Agent is an attribute of Anchor	Entity
		Key conflict	
		- Code → Name	Code

2b) Global conceptual schema



2c) Conceptual to logical translation

TVSHOW (Title, Date, Edition*, Duration*, Special)
 CASTMEMBER (Code, Firstname, Lastname, Address*, AgentPhoneNumber*)
 CAST (TvShowTitle, TvShowDate, CastMemberCode, AppearanceDuration*, EngagementFee)
 AGENT (PhoneNumber, Firstname*, Lastname*, Company*)
 ROLE (Name, Description)
 CASTROLE (CastMemberCode, RoleName)

3. Mapping definition and query answering

3a) Query formulation on the global schema

Select first name, last name and address of the cast members of TV shows that have been on air in special occasions.

```

SELECT DISTINCT CM.Firstname, CM.Lastname, CM.Address
FROM GS.CastMember AS CM, GS.Cast AS C, GS.TvShow AS T
WHERE CM.Code=C.CastMemberCode AND C.TvShowTitle=T.Title AND C.TvShowDate=T.Date AND
      T.Special=true
  
```


3b) GAV Mapping definition

NOTE: the KeyGen(,) functions are used to generate univocal identifiers.

NOTE: We suppose that the shows present in DS1 are disjoint from those present in DS2, or that duplicates due to possible overlapping between the two source databases are resolved in pre- or post-processing phases with which we do not deal. The same holds for cast members and agents.

```
CREATE VIEW GS.TvShow (Title, Date, Edition, Duration, Special) AS (  
    SELECT Title, Date, null, null, isSpecial(Date)  
    FROM DS1.Show  
    WHERE Type='TvShow'  
  
    UNION  
  
    SELECT T.Title, D.Date, T.Edition, T.Duration, D.Special  
    FROM DS2.TvShow AS T, DS2.Schedule AS S, DS2.Day AS D  
    WHERE T.Title=S.TvShowTitle AND T.Edition=S.TvShowEdition AND S.Date=D.Date  
)
```

NOTE: We assume the availability of a function isSpecial(date) determining whether a given date is a special day.

```
CREATE VIEW GS.CastMember (Code, Firstname, Lastname, Address, AgentPhoneNumber) AS (  
    SELECT KeyGenCastMember(CM.Code,'DS1'), CM.Firstname, CM.Lastname, CM.Address,  
        A.PhoneNumber  
    FROM DS1.CastMember AS CM, DS1.Agent AS A, DS1.Cast AS C, DS1.Show AS S  
    WHERE CM.AgentCode=A.Code AND CM.Code=C.CastMemberCode AND C.ShowTitle=S.Title  
        AND C.ShowDate=S.Date AND S.Type='TvShow'  
  
    UNION  
  
    SELECT KeyGenCastMember(Name, 'DS2'), left(Name, position(' $ ' in Name)-1), right(Name,  
        length(Name)-position(' $ ' in Name)), null, AgentPhoneContact  
    FROM DS2.Anchor  
  
    UNION  
  
    SELECT KeyGenCastMember(Name, 'DS2'), left(Name, position(' $ ' in Name)-1), right(Name,  
        length(Name)-position(' $ ' in Name)), Address, null  
    FROM DS2.Guest  
)
```

```

CREATE VIEW GS.Cast (TvShowTitle, TvShowDate, CastMemberCode, AppearanceDuration,
EngagementFee) AS (

    SELECT C.ShowTitle, C.Date, KeyGenCastMember(C.CastMemberCode,'DS1'),
        C.AppearanceDuration, CM.FeePerHour*C.AppearanceDuration
    FROM DS1.Cast AS C, DS1.CastMember AS CM, DS1.Show AS S
    WHERE C.CastMemberCode=CM.Code AND C.ShowTitle=S.Title AND C.Date=S.Date AND
        S.Type='TvShow'

    UNION

    SELECT A.TvShowTitle, S.Date, KeyGenCastMember(A.AnchorName, 'DS2'), null, A.EngagementFee
    FROM DS2.AnchoredBy AS A, DS2.Schedule AS S
    WHERE A.TvShowTitle=S.TvShowTitle AND A.TvShowEdition=S.TvShowEdition

    UNION

    SELECT TvShowTitle, TvShowDate, KeyGenCastMember(CastMemberName, 'DS2'), null,
        GuestEngagementFee
    FROM DS2.Schedule
)

```

NOTE: the following mappings (Agent, Role and CastRole) were not required by the exercise

```

CREATE VIEW GS.Agent (PhoneNumber, Firstname, Lastname, Company) AS (
    SELECT A.PhoneNumber, A.Firstname, A.Lastname, A.Company
    FROM DS1.Agent AS A, DS1.CastMember AS CM, DS1.Cast AS C, DS1.Show AS S
    WHERE A.Code=CM.AgentCode AND CM.Code=C.CastMemberCode AND C.TvShowTitle=T.Title
        AND C.TvShowDate=T.Date AND T.Type='TvShow'

    UNION

    SELECT AgentPhoneContact, null, null, null, null
    FROM DS2.Anchor
    WHERE AgentPhoneContact IS NOT NULL
)

CREATE VIEW GS.Role (Name, Description) AS (
    SELECT Name, Description
    FROM DS1.Role
)

```

```

CREATE VIEW GS.CastRole (CastMemberCode, RoleName) AS (
    SELECT KeyGenCastMember(CR.CastMemberCode, 'DS1'), CR.RoleName
    FROM DS1.CastRole AS CR, Cast AS C, Show AS S
    WHERE CR.CastMemberCode=C.CastMemberCode AND C.ShowTitle=S.Title AND

```

C.ShowDate=S.Date **AND** S.Type='TvShow'

UNION

SELECT KeyGenCastMember(Name, 'DS2'), 'stable-member'
FROM DS2.Anchor

UNION

SELECT KeyGenCastMember(Name, 'DS2'), 'guest'
FROM DS2.Guest

)

3c) Query rewriting on the data sources

Select first name, last name and address of the cast members of TV shows that have been on air in special occasions.

SELECT CM.Firstname, CM.Lastname, CM.Address
FROM DS1.CastMember **AS** CM, DS1.Cast **AS** C, DS1.Show **AS** S
WHERE CM.Code=C.CastMemberCode **AND** C.ShowTitle=S.ShowTitle **AND** C.ShowDate=S.Date **AND**
S.Type='TvShow' **AND** isSpecial(S.Date)=true

UNION

SELECT left(A.AnchorName, position('\$' in A.AnchorName)-1), right(A.AnchorName,
length(A.AnchorName)-position('\$' in A.AnchorName)), null
FROM DS2.AnchoredBy **AS** A, DS2.Schedule **AS** S, DS2.Day **AS** D
WHERE A.TvShowTitle=S.TvShowTitle **AND** A.TvShowEdition=S.TvShowEdition **AND** S.Date=D.Date **AND**
D.Special=true

UNION

SELECT left(G.Name, position('\$' in G.Name)-1), right(G.Name, length(G.Name)-position('\$' in G.Name)),
G.Address
FROM DS2.Guest **AS** G, DS2.Schedule **AS** S, DS2.Day **AS** D
WHERE G.Name=S.GuestName **AND** S.Date=D.Date **AND** D.Special=true

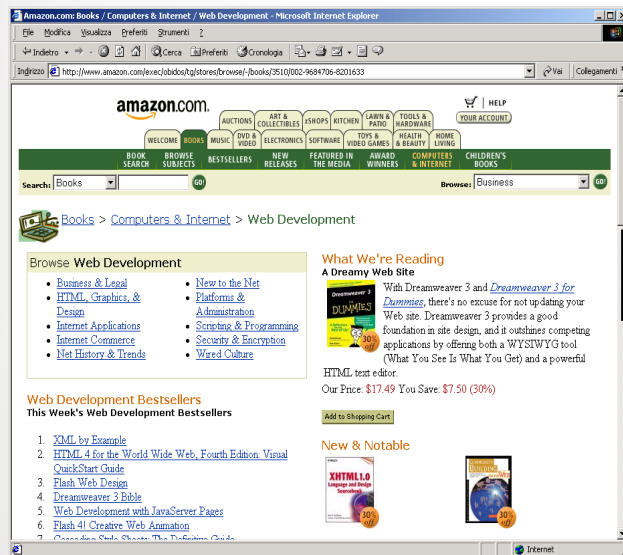
Typical complications when integrating semi- or un-structured data

- Each mediator is **specialized into a certain domain** (e.g. weather forecast), thus:
 - the mediator must know **domain metadata** ,
which convey the **data semantics**
- **On-line duplicate recognition, reconciliation and removal** (no designer to solve conflicts at design time here)
- If a data source changes a little, the wrapper has to be modified → **automatic wrapper generation**

LET'S GO BACK TO THE CONCEPT OF WRAPPER

- Wrappers **convert queries** into queries/commands which are understandable for the specific data source, but can also **extend** the query possibilities of a data source (e.g., may contain instructions for solving some data conflicts, like currency or measure-unit transformations)
- Wrappers **convert query results** from the source's format to a format which is understandable for the application

a WRAPPER for Web Pages



HTML page



BookTitle	Author	Editor
The HTML Sourcebook	J. Graham	...
Computer Networks	A. Tannenbaum	...
Database Systems	R. Elmasri, S. Navathe	...
Data on the Web	S. Abiteboul, P. Buneman, D. Suciu	...

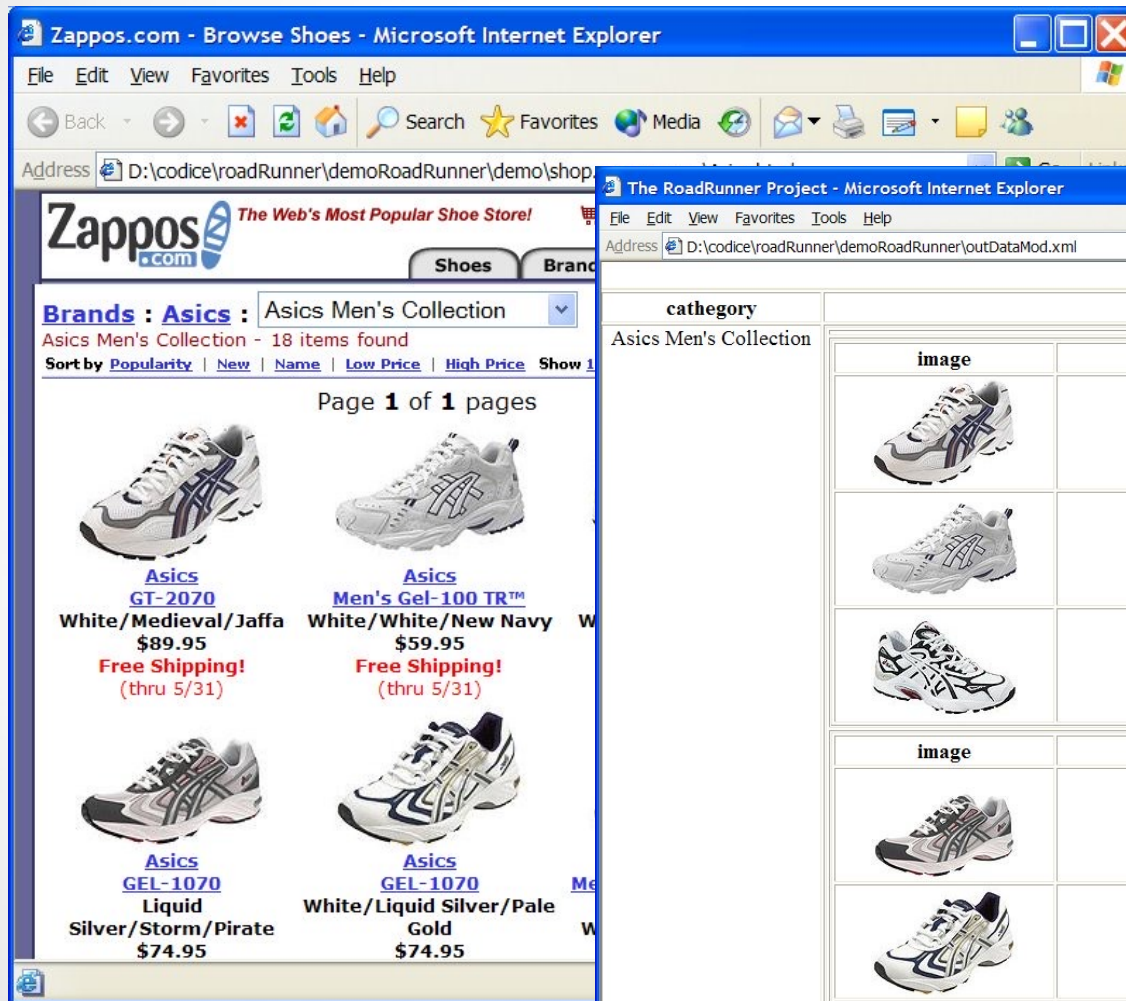
database table(s)
or another, preferred
model

Extraction of information from HTML docs

- Information extraction
 - Source Format: plain text with HTML tags (no semantics)
 - Target Format: e.g., relational table, XML, JSON, etc.(we add *structure*, i.e. *semantics*)
 - Much easier if the underlying page structure is derived from a DB
- Wrapper
 - Software module that performs an *extraction step*
 - Intuition: use extraction rules which exploit the *marking tags*

A complex extraction process

20-30KB IN HTML



The screenshot shows 'The RoadRunner Project' website in a Microsoft Internet Explorer browser. The address bar shows 'D:\codice\roadRunner\demoRoadRunner\outDataMod.xml'. The page displays a table of shoes under the category 'Asics Men's Collection'.

image	brand	model	descr	price
	Asics	GT-2070	White/Medieval/Jaffa	\$89.95
	Asics	Men's Gel-100 TR™	White/White/New Navy	\$59.95
	Asics	GEL-MC PLUS® V	White/White/Russet	\$99.95
image	brand	model	descr	price
	Asics	GEL-1070	Liquid Silver/Storm/Pirate	\$74.95
	Asics	GEL-1070	White/Liquid Silver/Pale Gold	\$74.95
	Asics	Men's GEL-Foundation III	White/Cinder/Blaze	\$79.95
image	brand	model	descr	price

>1000 items
with new

Problems

- Web sites change very frequently
- A layout change may affect the extraction rules
- Human-based maintenance of an ad-hoc wrapper is very expensive
- Better: ***automatic wrapper generation***

Automatic wrapper generation

- We can only use it when pages are *regular* to some extent
- OK when many pages share the same structure, e.g. pages are dynamically generated from a DB

→ *data intensive* web sites

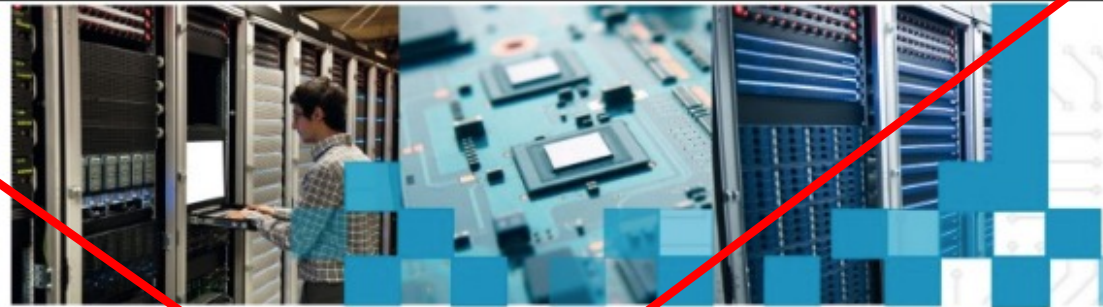
AN EXAMPLE OF SEMISTRUCTURED DATA THAT IS NOT FIT FOR AUTOMATIC WRAPPER GENERATION

Prof. Letizia Tanca

POLITECNICO DI MILANO

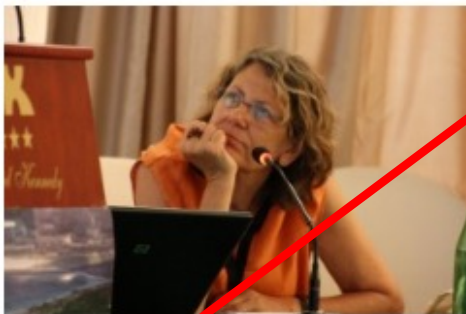


DIPARTIMENTO DI
ELETTRONICA,
INFORMAZIONE
E BIOINGEGNERIA



[Letizia Tanca](#) [Books](#) [Projects](#) [Publications](#) [Research Interests and Thesis Topics](#) [Teaching](#) [Contacts](#)

Letizia Tanca



Big Data and its 5 Vs – volume, velocity, variety, veracity, and value – have been talked about a lot. In their **ACM SIGMOD blog post**, Letizia Tanca is writing about the **Double Y for Big Data: Wisdom**. Reciting an extract from her blog: *...not only we want to*

Recent Comments

Archives

Categories

- No categories

Meta

- [Log in](#)
- [Entries RSS](#)
- [Comments RSS](#)
- [WordPress.org](#)

YES!!

SEMISTRUCTURED DATA:

a page produced from a database

[ITA](#) [ENG](#)

[SCHOOL@DEIB](#) [I4.0](#) [HONOURS PROGRAM](#) [PEOPLE@DEIB](#) [EVENTI DEIB](#) [INTRANET](#)

**POLITECNICO**
MILANO 1863
DIPARTIMENTO DI ELETTRONICA
INFORMAZIONE E BIOINGEGNERIA

[HOME](#) [NOTIZIE ED EVENTI](#) [CHI SIAMO](#) [RICERCA](#) [INDUSTRIA](#) [PERSONALE](#) [RISORSE](#) [DIDATTICA](#)

» *Personale*

[Prof. TANCA LETIZIA](#)

Professore Ordinario





Sede: Edificio 20
Piano: 1°
Ufficio: 050
Tel.: 3531

 letizia.tanca@polimi.it



Area di ricerca:
» [Informatica](#)

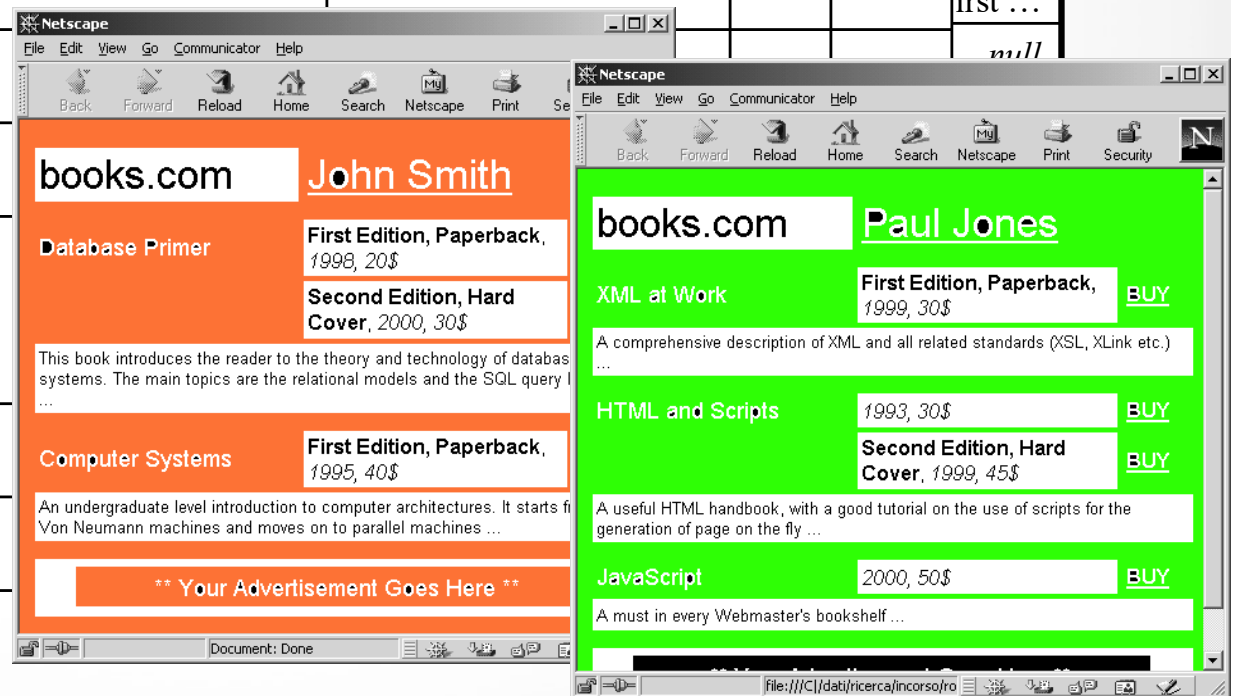
Linea di ricerca:
» [Dati, web e società](#)



Pagina personale
<http://tanca.faculty.polimi.it>

Example: an online library

Name	Books					Details
	Title	Description	Editions			
			Details	Year	Price	
John Smith	Database Primer	This book ...	First Edition, Paperback	1998	20\$	first ...
			Second Edition, Hard Cover	2000	30\$	second ...
	Computer Systems					first ...
						first ...
Paul Jones	XML at Work					first ...
	HTML and Scripts					first ...
						first ...
	JavaScripts					first ...
...	...	...				first ...



Source Dataset

HTML Pages

The ROAD-RUNNER project

- Page Class
 - *It is the collection of all the pages generated by the same script from a common dataset*
 - Schema Derivation
 - *Given a set of HTML sample pages, belonging to the same class, find the underlying dataset structure (database schema)*
- ➔ Solution: Wrapper Generator
- Underlying dataset structure
 - Extraction rules

The new application context (recall)

- A (possibly large) number of data sources
- Heterogeneous data sources
- Different levels of data structure
 - Databases (relational, OO...)
 - Semi-structured data sources (XML, HTML, more markups ...)
 - Unstructured data (text, multimedia etc...)
- Different terminologies and different operational contexts
- Time-variant data (e.g. WEB)
- Mobile, transient data sources

We need tools for recognizing DATA SEMANTICS: Ontologies

- An ONTOLOGY is a formal and shared definition of a vocabulary of terms and their inter-relationships
- Predefined relations:
 - *synonymy*
 - *onymy*
 - *hyponymy*
 - *etc..*
- More complex, designer-defined relationships, whose semantics depends on the domain

e.g. `enrolled(student, course)`

→ an ER diagram, a class diagram, any conceptual schema *is a kind of ontology!*

Definitions

- Ontology = formal specification of a conceptualization of a shared knowledge domain.
- An ontology is a controlled vocabulary that describes objects and the relationships between them in a formal way
- It has a grammar for using the terms to express something meaningful within a specified domain of interest.
- The vocabulary is used to express queries and assertions.
- Ontological commitments are agreements to use the vocabulary in a consistent way for knowledge sharing

Aims...

- A formal specification allows for use of a common vocabulary for *automatic knowledge sharing*
- Formally specifying a *conceptualization* means *giving a unique meaning* to the terms that define the knowledge about a given domain
- *Shared*: an ontology captures knowledge which is common, thus *over which there is a consensus* (objectivity is not an issue here)

Ontology types

- **Taxonomic ontologies**

- Definition of concepts through terms, their hierarchical organization, and additional (pre-defined) relationships (synonymy, composition,...)
- To provide a reference vocabulary

- **Descriptive ontologies**

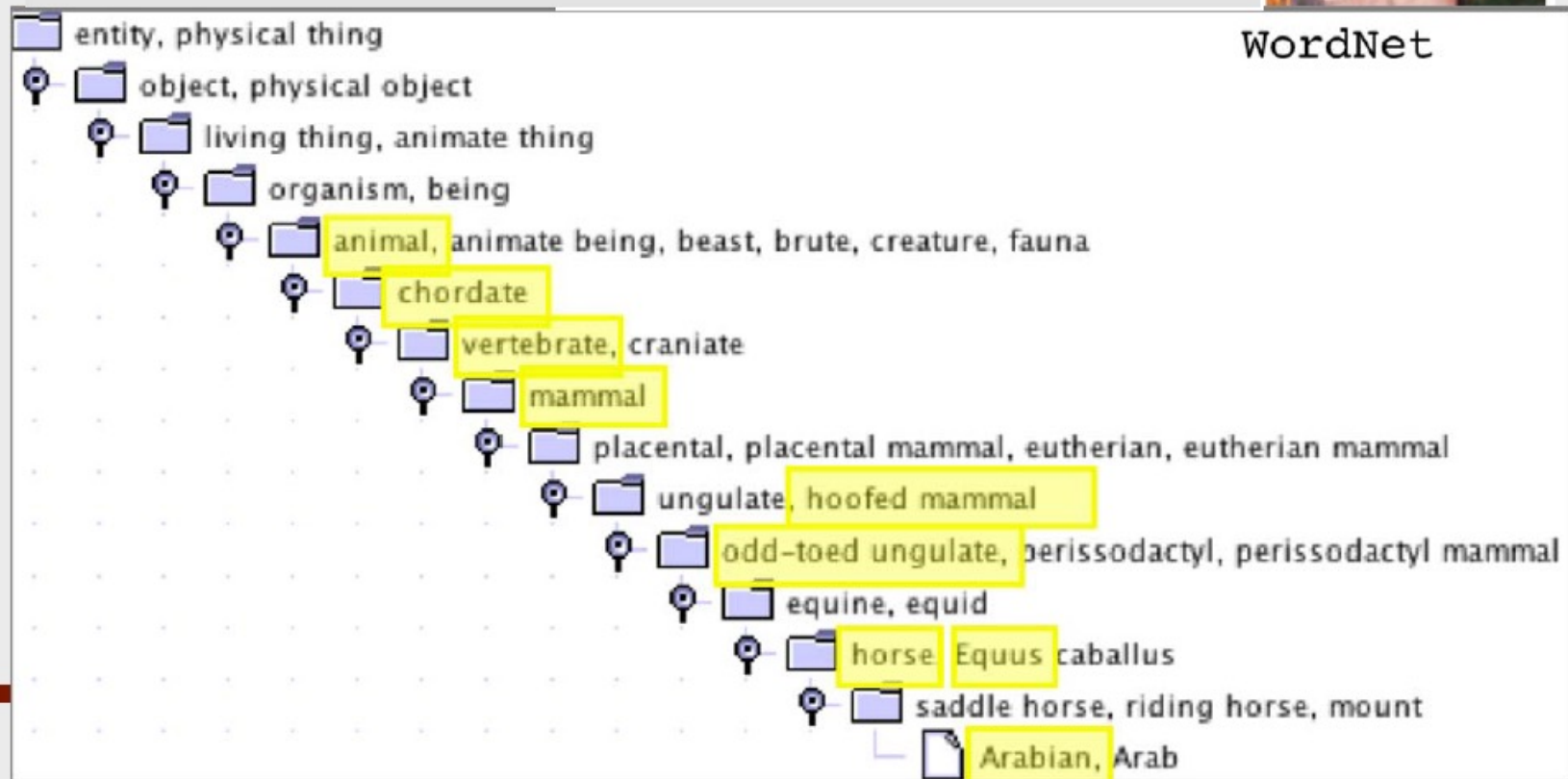
- Definition of concepts through data structures and their interrelationships
- Provide information for “aligning” existing data structures or to design new, specialized ontologies (**domain ontologies**)
- Closer to the database area techniques

Wordnet: a taxonomic ontology

horse, Equus caballus:
a solid-hoofed
herbivorous quadruped
domesticated since
prehistoric times



WordNet



An ontology consists of...

- **Concepts:**
 - Generic concepts, they express general world categories
 - Specific concepts, they describe a particular application domain (domain ontologies)
- **Concept Definition:**
 - Via a formal language
 - In natural language
- **Relationships between concepts:**
 - Taxonomies (IS_A),
 - Meronymies (PART_OF),
 - Synonymies, homonymies, ...
 - User-defined associations,

Formal Definitions

$$O = (C, R, I, A)$$

O: ontology, C: concepts, R: relations, A: axioms,
I: Instances

- Often specified in some logic-based language
- Organized in a ISA hierarchy
- *I* is an instance collection, stored in the information source

MORE IN THE NEXT LECTURE

Bibliography

- A. Doan, A. Halevy and Z. Ives, Principles of Data Integration, Morgan Kaufmann, 2012
- L. Dong, D. Srivastava, Big Data Integration, Morgan & Claypool Publishers, 2015
- Roberto De Virgilio, Fausto Giunchiglia, Letizia Tanca (Eds.): Semantic Web Information Management – A Model-Based Perspective. Springer 2009, ISBN 978-3-642-04328-4
- M. Lenzerini, Data Integration: A Theoretical Perspective, Proceedings of ACM PODS, pp. 233-246, ACM, 2002, ISBN: 1-58113-507-6
- Clement T. Yu, Weiyi Meng, Principles of Database Query Processing for Advanced Applications , Morgan Kaufmann, 1998, ISBN: 1558604340